Software Engineering (Project A)

Authors

Riccardo Masarin	10708900
Manuel Mannucci	10761728
Alessandro Forlani	10753247

Design Document

POLITECNICO
MILANO 1863

MSc Automation and Control Engineering

July 24, 2026

Contents

1 Introduction **1**
1.1 Purpose . . . 1
1.2 Definitions, Acronyms and Abbreviations . . . 1
1.3 Revision History . . . 2
1.4 Document Structure . . . 2

2 Architectural Design **3**
2.1 Overview . . . 3
2.2 Component View . . . 5
2.3 Class View . . . 8
2.3.1 Core Domain Class Diagram . . . 8
2.3.2 Ticket State Pattern . . . 9
2.3.3 Recommendation Strategy Pattern . . . 10
2.3.4 Supporting Entities and Infrastructure Mechanisms . . . 11
2.4 Runtime View . . . 11
2.4.1 Ticket Request and Confirmation . . . 12
2.4.2 Ticket Cancellation and Waiting-List Promotion . . . 13
2.4.3 Venue Manager Creates an Event . . . 13
2.4.4 User Receives Recommended Events . . . 14
2.4.5 Other Runtime Interactions . . . 15
2.5 Selected Architectural Styles and Patterns . . . 15

3 User Interface Design **17**
3.1 UI Overview . . . 17
3.2 Navigation and Role-Based Routing . . . 17
3.3 Representative Interfaces . . . 18
3.3.1 Discovery Feed . . . 19
3.3.2 Event Detail and Ticket Action . . . 19
3.3.3 Pregame Room . . . 20
3.3.4 User Profile . . . 20
3.3.5 Venue Manager Dashboard . . . 21
3.3.6 PR Dashboard . . . 21

4 Requirements Traceability **22**
4.1 Traceability Approach and Status Definitions . . . 22
4.2 Functional Requirements Traceability . . . 23
4.2.1 User and Profile Requirements . . . 23
4.2.2 Event Discovery Requirements . . . 23
4.2.3 Recommendation Requirements . . . 24
4.2.4 Reservation and Ticketing Requirements . . . 24
4.2.5 Pregame Requirements . . . 25
4.2.6 Social Layer Requirements . . . 25
4.2.7 Notification Requirements . . . 26
4.2.8 Venue Manager Requirements . . . 26
4.3 Traceability Summary . . . 27

5 Implementation, Integration and Test Plan **27**

5.1 Implementation Order 27
5.2 Integration Plan 28
5.3 Testing Strategy 29
5.3.1 Tests Currently Implemented 30
5.3.2 Main Tested Behaviors 32
5.3.3 Build and Test Status 33
5.3.4 Relation to Verification and Validation Theory 33
5.3.5 Remaining Automated Testing Plan 34
5.4 Test Data 34

1 Introduction

1.1 Purpose

This document refines the requirements defined in the Requirements Analysis and Specification Document and describes how the system is currently structured. In particular, it presents the software architecture, the main components, the relevant class diagrams, the runtime interactions among components, the user interface design, the mapping between requirements and design elements, and the implementation, integration, and testing plan.

In particular, the document presents:

- the client-server and layered architecture of the system;
- the main frontend and backend components;
- the implemented domain model and persistence structure;
- the design patterns effectively used at runtime;
- the main interactions between the React frontend and the Spring Boot backend;
- the implemented user, venue manager, and PR manager interfaces;
- the traceability between functional requirements and implementation elements;
- the current implementation, integration, testing, and dataset status;
- the main prototype limitations and known implementation differences.

NightOUT is implemented as a React and TypeScript web client communicating through REST APIs with a Java Spring Boot backend. The backend uses Spring Data JPA and an in-memory H2 database for persistence during the execution of the prototype.

1.2 Definitions, Acronyms and Abbreviations

User
A registered person using the system to discover events, reserve spots, join pregame rooms, and interact with social features.

Venue
A physical place, such as a club, bar, lounge, or live music venue, where nightlife events take place.

Venue Manager
A user whose role is `VENUE_MANAGER`. Venue managers can manage their assigned venues, create and update events, manage promotions, inspect tickets, and access dashboard information.

PR Manager
A user whose role is `PR_MANAGER`. PR managers can access assigned event information, attributed ticket data, commissions, and PR-specific dashboard information.

Event
A nightlife activity associated with a venue.

Ticket
A persistent record representing a user's request to attend an event. A ticket progresses

through a lifecycle that may include pending, confirmed, waiting-list, cancelled, and expired states. Confirmed tickets contain a digital QR payload used by the prototype as an admission confirmation.

Waiting List
A list of users waiting for a spot in an event that reached its full capacity.

Pregame Room
A temporary group linked to an event, used by users to coordinate before attending the event.

Friendship
A social relationship between two users created through a request workflow. A friendship request may be pending, accepted, or rejected.

Notification
A message generated by the system to inform a user about relevant updates.

REST API
Representational State Transfer Application Programming Interface.

DTO
Data Transfer Object.

UI
User Interface.

1.3 Revision History

Version	Date	Description
1.0	Initial design proposal	Initial planned software architecture and implementation proposal
1.1	Updated working	Long as-built working version
2.0	Final	Final summarized version with the current React, Spring Boot, JPA and H2 implementation

1.4 Document Structure

Section 1 introduces the purpose, terminology, revision history, and structure of the document.

Section 2 describes the as-built architecture of NightOUT, including the client-server structure, backend layers, implemented components, domain classes, runtime interactions, and design patterns.

Section 3 presents the current user interfaces and route organization for normal users, venue managers, and PR managers.

Section 4 provides the requirements traceability matrix, that maps the functional requirements defined in the Requirements Analysis and Specification Document to the corresponding implementation elements and reports their implementation status.

Section 5 describes the current implementation and integration status, the automated tests actually available, the additional tests still required, the prototype dataset, and the main known limitations.

Section 6 lists the references used in the document.

2 Architectural Design

2.1 Overview

NightOUT follows a **client-server architecture**.

The client side is a mobile-oriented React and TypeScript frontend. It is responsible for presentation, navigation, user interaction, and local interface state. The frontend communicates with the backend through an Axios-based REST API client and exchanges JSON Data Transfer Objects.

Three main roles access the application:

- normal users;
- venue managers;
- PR managers.

The frontend does not contain the authoritative application data. Operational information concerning users, venues, events, tickets, pregame rooms, friendships, promotions, and notifications is retrieved from the backend through REST API calls.

The server side is implemented in Java using Spring Boot. It exposes the REST endpoints, validates incoming requests, executes the application and domain logic, coordinates persistence operations, and returns DTO responses to the frontend.

The database stores users, venue managers, venues, events, reservations, pregame rooms, promotions, notifications, social relations, and event participation information.

The backend follows a layered architecture:

1. **REST Controller Layer**
   Receives HTTP requests, validates request DTOs, invokes the appropriate application services, and returns response DTOs.
2. **Application Service Layer**
   Implements use-case management, transaction boundaries, business rules, and coordination among domain objects, recommendation strategies, notification mechanisms, and persistence components.
3. **Domain and Business Behavior Layer**
   Contains the JPA entities, enums, ticket lifecycle states, recommendation strategies, and domain-related events used by the application.
4. **Data Mediation Layer**
   Contains multiple focused data mediators. Each mediator coordinates the repositories required by a specific application area, such as tickets, events, users, pregames, recommendations, or manager dashboards.

5. **Repository Layer**
   Contains Spring Data JPA repositories responsible for persistent entity access, queries, counts, and data updates.

6. **Persistence Layer**
   Uses an in-memory H2 relational database. Hibernate is configured with a create-drop policy, meaning that data remains available while the backend process is running but is recreated when the backend restarts.

Application services also collaborate with several specialized runtime components. The ticket lifecycle is implemented through concrete State objects selected by a state factory. Personalized recommendations are produced by a recommendation engine that evaluates multiple Spring-managed recommendation strategies. Ticket and friendship notifications use Spring application events and listeners, while some pregame notification operations currently invoke the notification service directly.

The current implementation contains two limited exceptions to the main data-access direction. `AuthenticationService` accesses `AppUserRepository` directly, while `NightOutMapper` performs additional repository queries when constructing aggregate event DTOs.

Most persistent application data is stored through JPA in H2. Avatar binary files are stored in the local file system, while their corresponding resource URLs are stored in the user entity.

Authentication in the current prototype is intentionally lightweight. The backend verifies the submitted credentials, while the frontend stores a session marker in browser `localStorage`. The prototype does not currently use Spring Security, server-side sessions, or token-based authentication. Consequently, backend operations still rely on user and manager identifiers supplied by the client.

The general architecture is shown below.

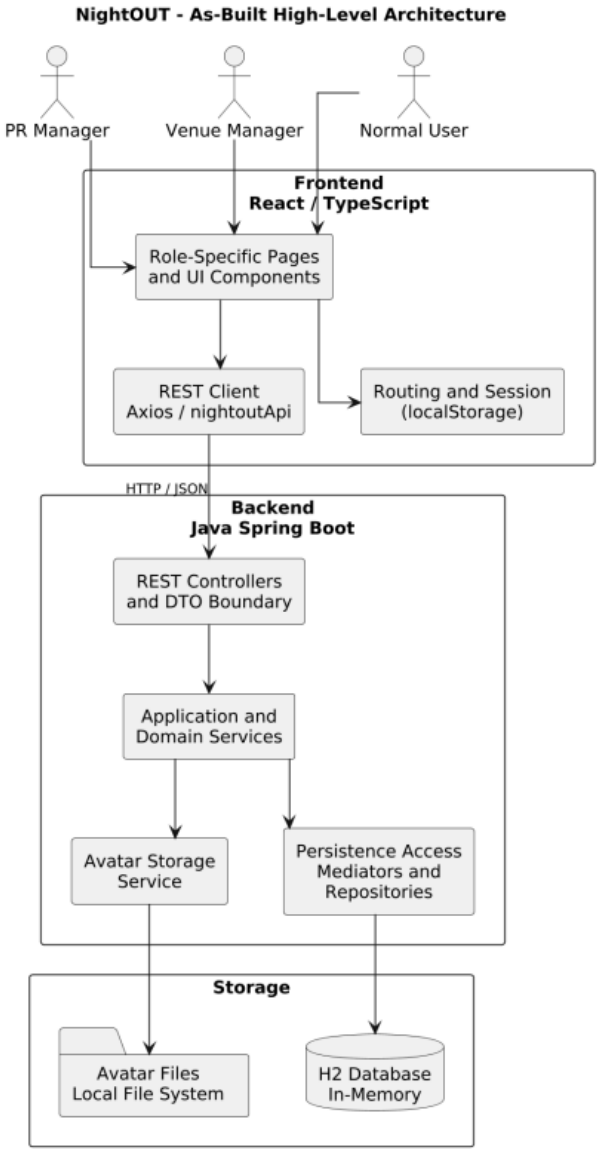


Figure 1: NightOUT - As-Built High-Level Architecture

2.2 Component View

The component view describes the main functional areas of the NightOUT implementation and the dependencies among the frontend, backend application components, data-access infrastructure, and persistence layer.

A component in this view represents a cohesive group of controllers, services, domain objects, mediators, repositories, and frontend pages that collaborate to provide a specific group of functionalities. Therefore, each component does not necessarily correspond to a single Java class or React component.

The React frontend is organized into role-specific pages for normal users, venue managers, and PR managers. All frontend areas communicate with the backend through the shared `nightoutApi` Axios client.

The Spring Boot backend is divided into the following main application components:

1. Authentication and User Account Component;
2. Event Discovery Component;
3. Venue and Event Administration Component;
4. Ticketing Component;
5. Pregame Component;
6. Friendship and Social Component;
7. Notification Component;
8. Recommendation Component;
9. Promotion and PR Component;
10. Data Access and Persistence Component.

The backend application components primarily access persistent data through focused data mediators. The mediators coordinate Spring Data JPA repositories, which store the application entities in the H2 database.

Some components also collaborate through specialized runtime mechanisms. Ticket lifecycle operations use the State pattern, recommendation scoring uses multiple Strategy implementations, and ticket and friendship notifications use Spring application events and listeners.

The component diagram is shown below.

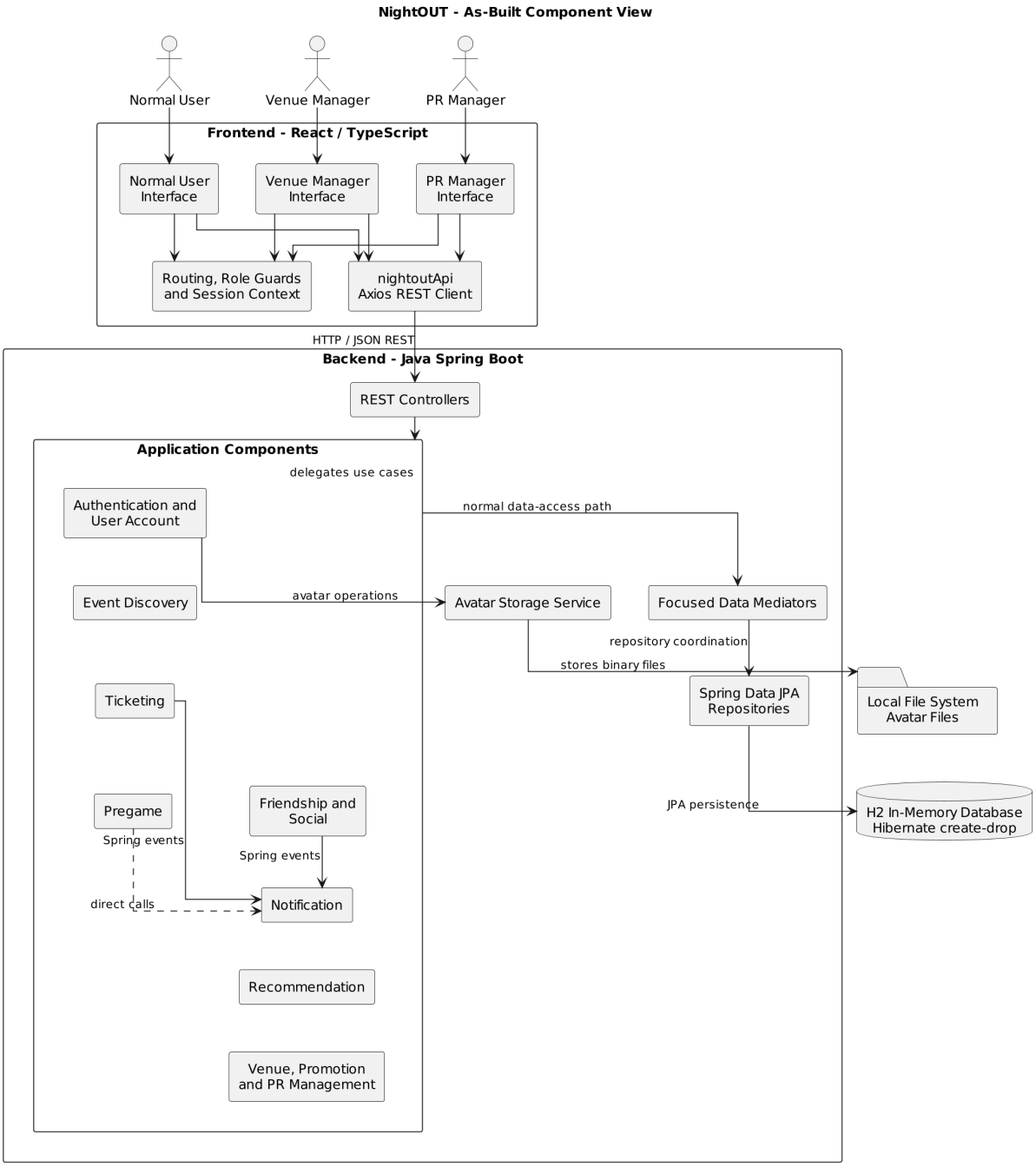


Figure 2: NightOUT - As-Built Component View

Component	Main responsibility	Main interfaces/classes
Authentication and User Account	Login, profile and preferences	`AuthController`, `UserService`
Event Discovery	Browsing, searching and filtering	`EventController`, `EventService`
Ticketing	Ticket lifecycle and waiting list	`TicketController`, `TicketService`
Pregame	Room creation and membership	`PregameController`, `PregameService`
Social and Notifications	Friendships and user updates	`FriendshipService`, `NotificationService`
Recommendation	Personalized event ranking	`RecommendationEngine`, strategies
Venue/PR Management	Events, venues, promotions and dashboards	manager and PR services
Persistence	Repository coordination and H2 storage	mediators, repositories

2.3 Class View

The class view presents the principal persistent entities and the two most relevant behavioral class structures of the current NightOUT implementation.

Most JPA entities primarily represent persistent state and relationships, while application services coordinate use cases and business rules. The main exception is the ticket lifecycle, whose behavior is delegated to runtime State objects. Personalized recommendations are instead implemented through multiple interchangeable Strategy components.

To preserve readability, only architecturally relevant attributes, relationships, and operations are shown. Secondary persistence entities and infrastructure collaborations are summarized without dedicated diagrams.

2.3.1 Core Domain Class Diagram

The NightOUT domain model is centred on users, venues, events, tickets, pregame rooms, friendships, and promotions.

`AppUser` represents every application account. Normal users, venue managers, and PR managers are distinguished through the `UserRole` attribute rather than through separate subclasses. A `Venue` is assigned to a manager and hosts multiple `Event` entities. Each event is also associated with the user who created it.

`Ticket` connects a user to an event and stores the persistent ticket type, price, lifecycle status, confirmation deadline, and digital confirmation payload. `PregameRoom` belongs to an event, has one host, and maintains a many-to-many collection of participants.

`Friendship` represents a directed friendship request between two users and stores its current status. A `Promotion` is associated with a venue and may optionally target a specific event.

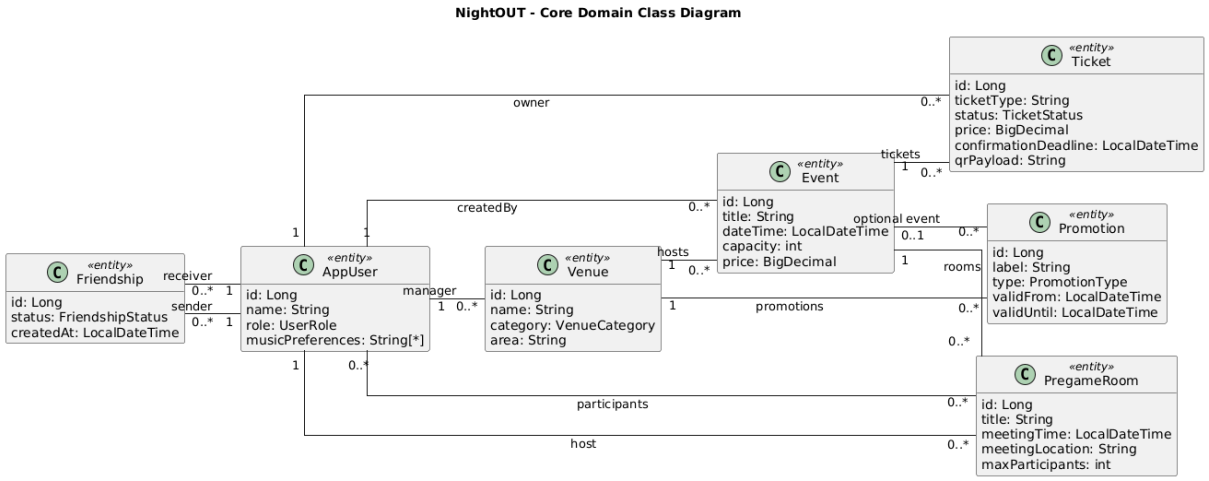


Figure 3: NightOUT - Core Domain Class Diagram

2.3.2 Ticket State Pattern

The ticket lifecycle is implemented through the State pattern. The persistent `Ticket` entity stores a `TicketStatus`, while lifecycle operations are delegated to a runtime implementation of `TicketState`.

`TicketService` coordinates ticket requests, capacity checks, confirmation, cancellation, expiration, and waiting-list promotion. `TicketStateFactory` selects the appropriate concrete State according to the ticket's current persistent status.

The five concrete states correspond to `PENDING`, `CONFIRMED`, `WAITING_LIST`, `CANCELLED`, and `EXPIRED`. Each state determines which lifecycle operations are valid and applies the corresponding status transition. The `Ticket` entity additionally validates the permitted transition matrix before persisting the new status.

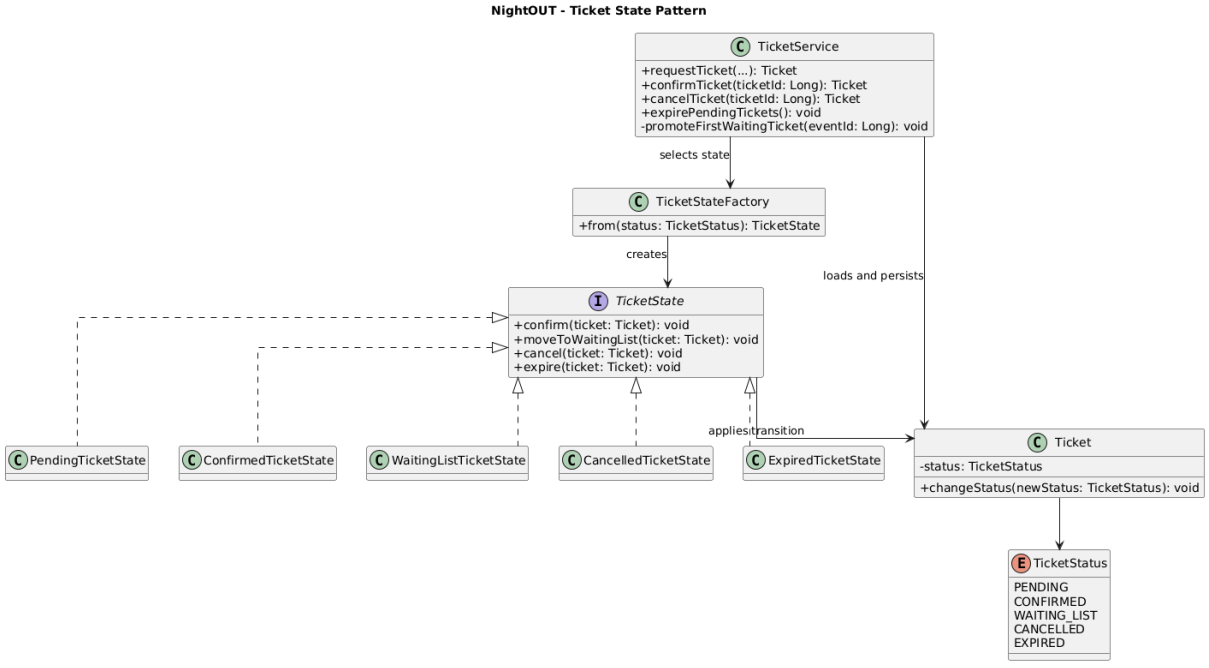


Figure 4: NightOUT - Ticket State Pattern

2.3.3 Recommendation Strategy Pattern

Personalized event ranking is implemented through the Strategy pattern. `RecommendationStrategy` defines the common contract used to evaluate one aspect of the relationship between a user and a candidate event.

The six implemented strategies evaluate music preferences, friends attending, geographic distance, booking history, saved events, and stored popularity. The strategies are registered as Spring components and injected into `RecommendationEngine`.

For each candidate event, the engine evaluates all applicable strategies and combines their score contributions and explanation reasons. `RecommendationService` retrieves the required information through `RecommendationDataMediator`, excludes unsuitable events, invokes the engine, and returns the ordered recommendations.

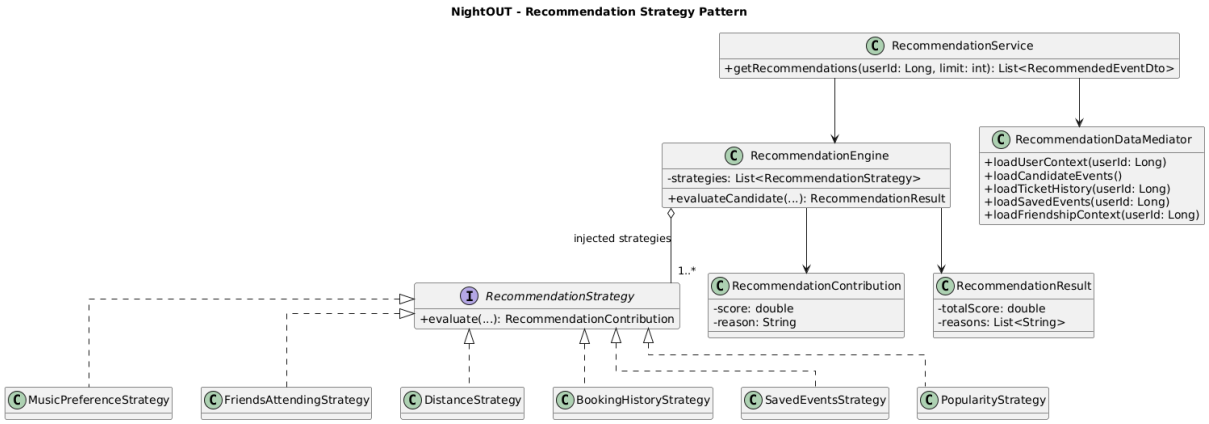


Figure 5: NightOUT - Recommendation Strategy Pattern

2.3.4 Supporting Entities and Infrastructure Mechanisms

Entity	Responsibility
`EventParticipation`	Stores saved-event information; confirmed attendance is derived from tickets.
`UserNotification`	Persists ticket, friendship, waiting-list, and pregame notifications.
`PrEventAssignment`	Associates PR managers with events and commission information.
`PaymentMethod`	Stores prototype payment-method information.
`SupportRequest`	Persists help and support messages submitted by users.
`ReturnTransportOption`	Stores optional return-transport information associated with events.
`SalesChannel`	Stores synthetic sales and revenue indicators used by manager dashboards.
`SocialRelation`	Legacy entity retained in the codebase but not used by the current friendship workflow.

Ticket and friendship notifications use Spring application events and listener components, providing Observer-style publish-subscribe behavior. The listeners invoke `NotificationService`, while pregame operations currently call the notification service directly. Since this mechanism is already represented in the component and pattern views, no separate class diagram is required here.

Application services normally access repositories through focused data mediators associated with their functional area. `AuthenticationService`, which accesses `AppUserRepository` directly, and `NightOutMapper`, which performs additional repository queries while constructing aggregate DTOs, are limited exceptions to this dependency direction.

2.4 Runtime View

The runtime view presents the interactions involved in the most architecturally significant NightOUT use cases. The selected sequence diagrams focus on workflows that cross the frontend, REST, application, domain-behavior, and persistence boundaries.

The normal execution path is:
React page → REST controller → application service → focused data mediator → repository and persistence.

Some use cases additionally involve runtime State objects, recommendation strategies, and Spring application events. Communication between the frontend and backend uses JSON request and response DTOs.

The current prototype does not derive the acting user from a server-side authenticated principal. User and manager identifiers are therefore supplied by the frontend through request parameters or DTO fields.

2.4.1 Ticket Request and Confirmation

Ticket acquisition is implemented as a two-step workflow.

First, the frontend sends a ticket request through `POST /api/tickets`. `TicketService` retrieves the selected user and event, verifies that no active ticket already exists, creates a `PENDING` ticket, and persists it through `TicketDataMediator`.

The user then confirms checkout through `POST /api/tickets/{ticketId}/confirm`. `TicketStateFactory` selects `PendingTicketState`, while the service retrieves the number of confirmed tickets for the event. The State changes the ticket to `CONFIRMED` when capacity is available or to `WAITING_LIST` when the event is full.

The updated ticket is persisted and a Spring application event triggers the corresponding user notification. The resulting status is returned to the frontend.

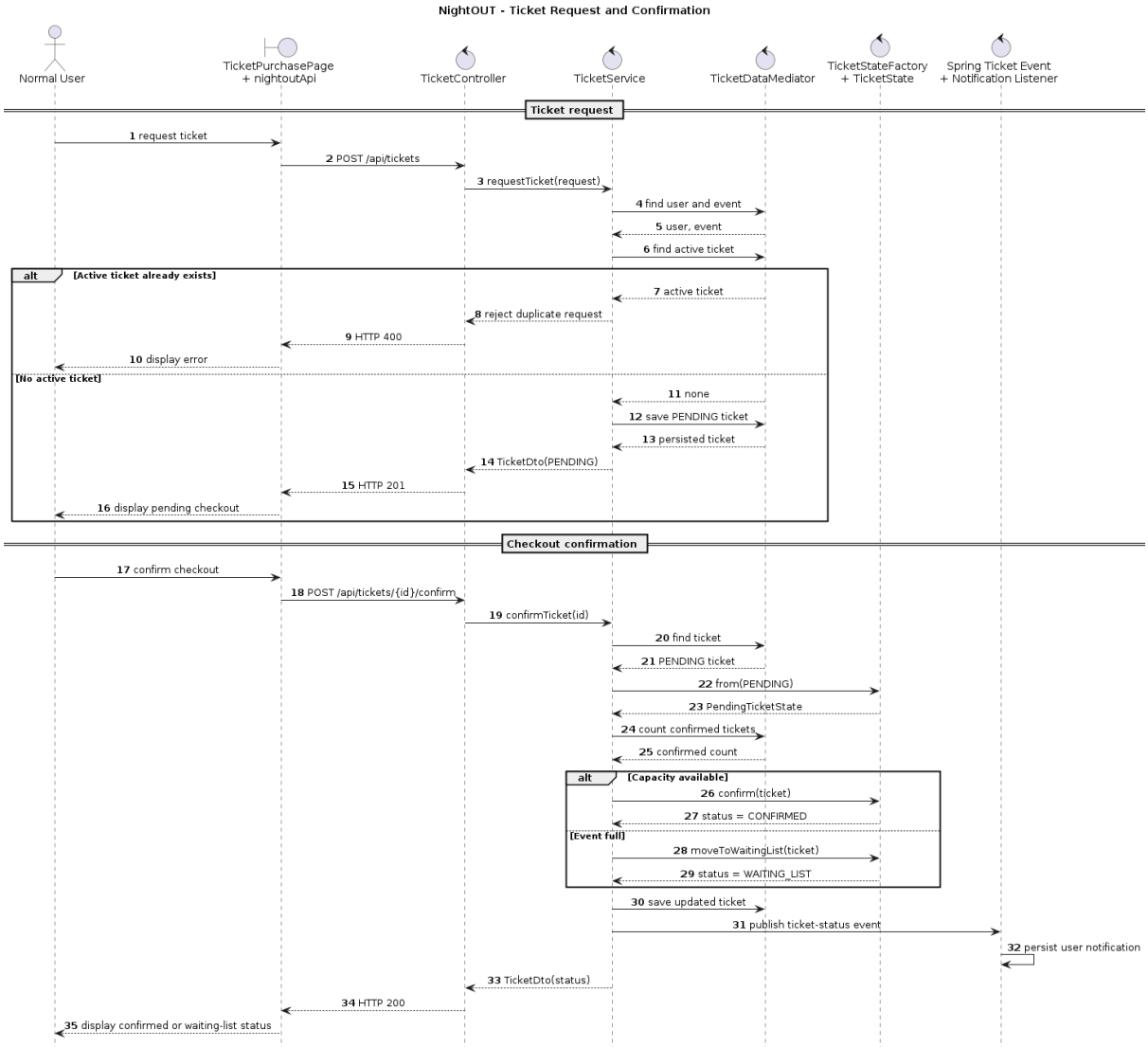


Figure 6: NightOUT - Ticket Request and Confirmation

2.4.2 Ticket Cancellation and Waiting-List Promotion

When an active ticket is cancelled, `TicketService` retrieves it and delegates the lifecycle transition to the State associated with its current status.

After persisting the cancelled ticket, the service retrieves the waiting list for the same event ordered by ticket creation time. When the list is not empty, the oldest ticket is selected and `WaitingListTicketState` promotes it to `CONFIRMED`.

Both cancellation and promotion generate Spring application events, from which the corresponding notifications are created.

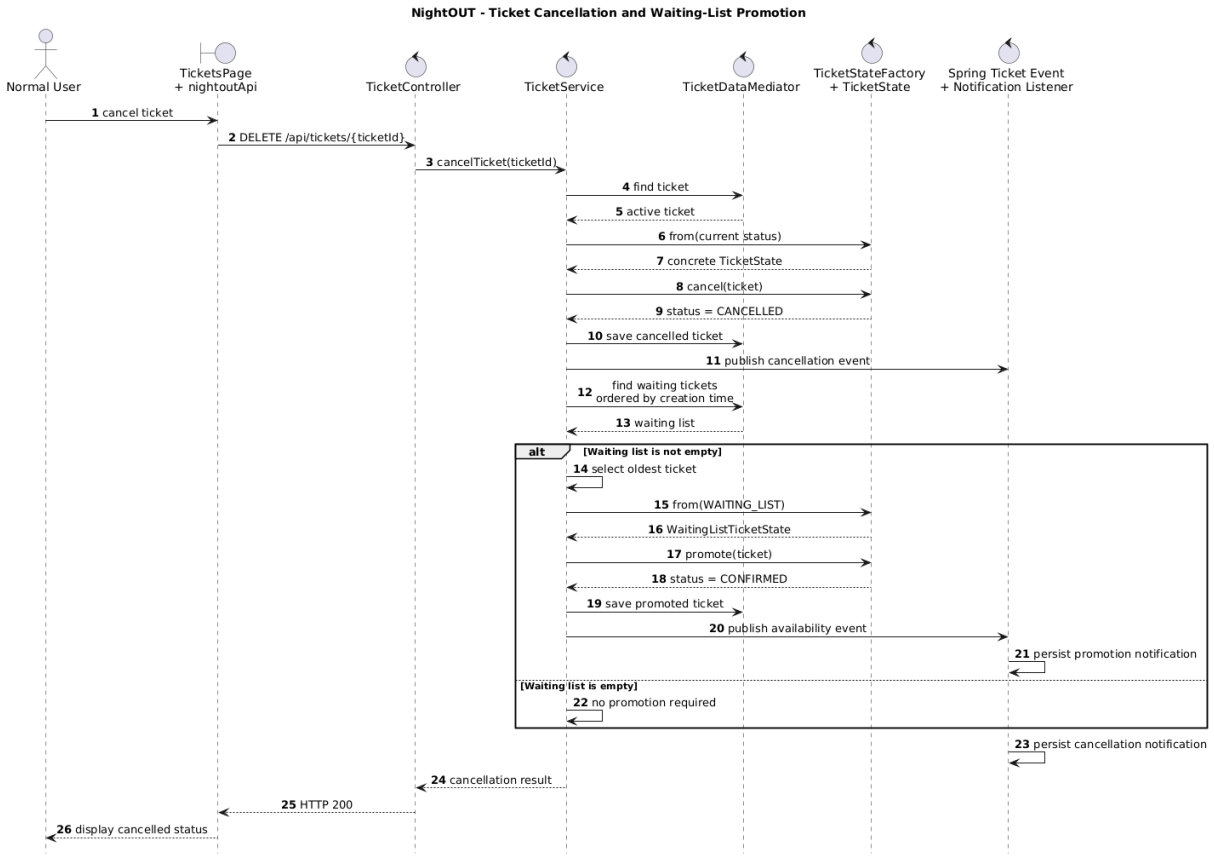


Figure 7: NightOUT - Ticket Cancellation and Waiting-List Promotion

2.4.3 Venue Manager Creates an Event

A venue manager creates an event through `ManagerCreateEventPage`, which submits a `CreateEventDto` to `POST /api/manager/events`.

`VenueManagementService` retrieves the supplied manager and venue, verifies the manager role, validates the event data, and persists the new Event through `VenueManagementDataMediator`.

Manager administration and normal-user discovery access the same `EventRepository`. Consequently, a successfully created event becomes available through the normal-user event feed.

The current implementation verifies the manager role but does not consistently verify that the selected venue belongs to that manager.

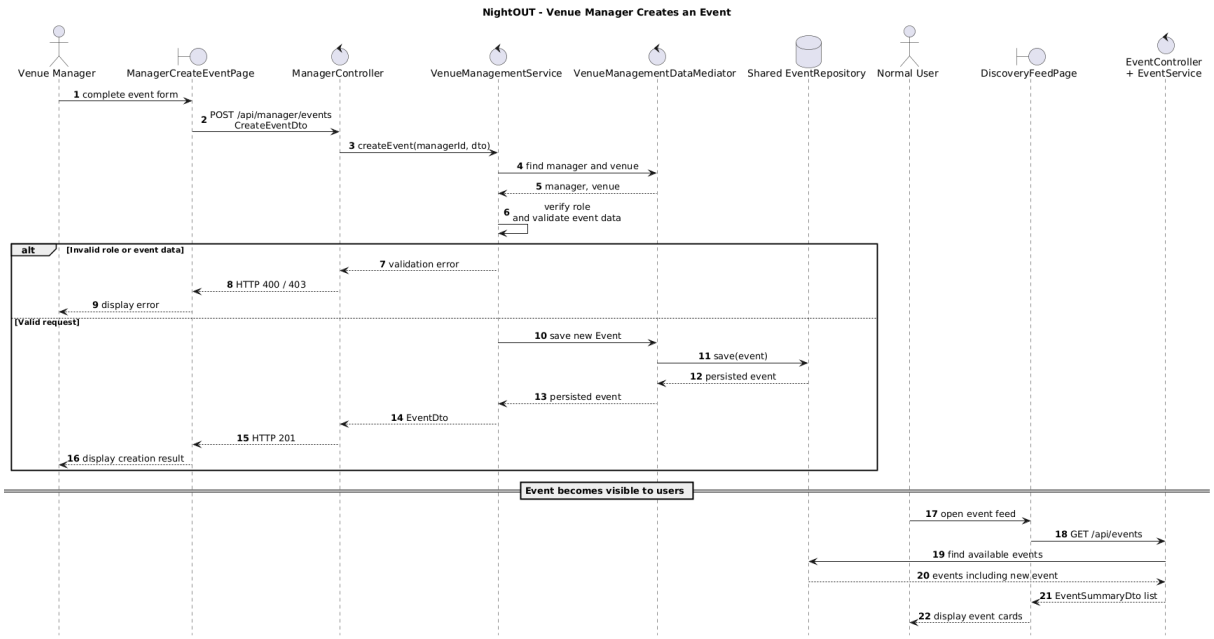


Figure 8: NightOUT - Venue Manager Creates an Event

2.4.4 User Receives Recommended Events

Personalized recommendations are requested by the normal-user feed.
`RecommendationService` obtains the user profile, candidate events, ticket history, saved events, and friendship information through `RecommendationDataMediator`. The resulting context is passed to `RecommendationEngine`.

For every candidate event, the engine evaluates all applicable implementations of `RecommendationStrategy`. Their score contributions and explanation reasons are combined into a single result.

Unsuitable or already attended events are excluded, while the remaining results are ordered and limited before being returned to the frontend.

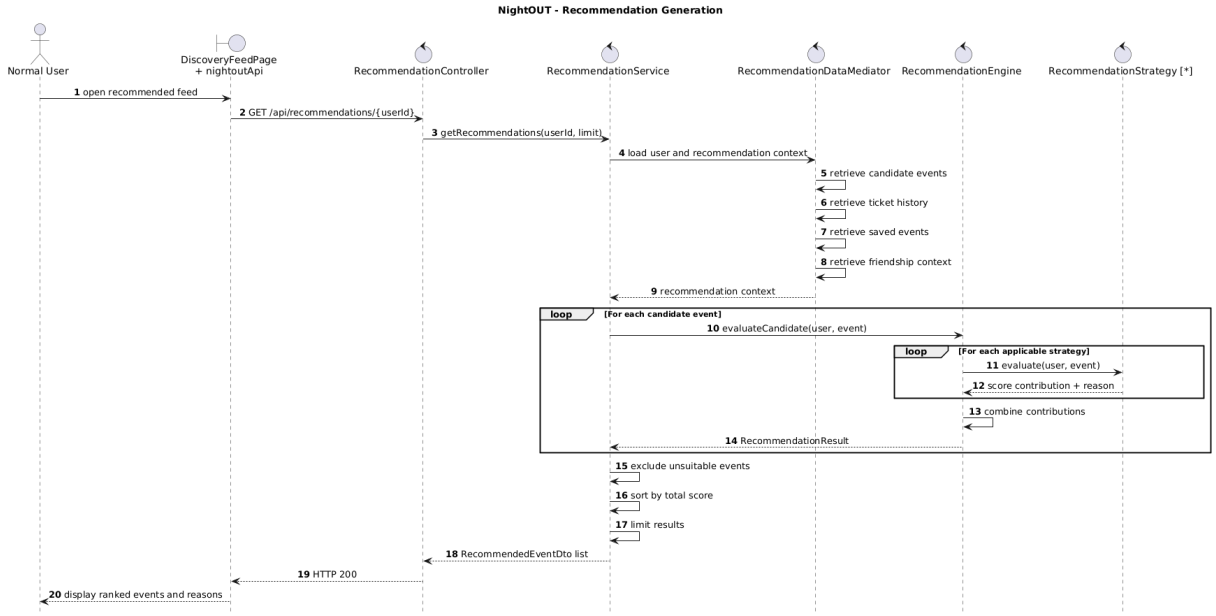


Figure 9: NightOUT - Recommendation Generation

2.4.5 Other Runtime Interactions

Event searching and filtering follows the standard frontend-controller-service-mediator execution path. `EventService` retrieves the available events and applies the selected filters and sorting criteria before returning event DTOs to the feed.

Pregame membership follows the same layered path. `PregameService` verifies room capacity and existing membership before adding or removing a participant and persisting the updated room. Pregame notifications currently invoke `NotificationService` directly rather than using Spring application events.

The manager dashboard aggregates information through `DashboardService` and `DashboardDataMediator`. Since this workflow mainly consists of data retrieval and prototype-specific aggregation, it is not represented by a dedicated sequence diagram.

2.5 Selected Architectural Styles and Patterns

The current NightOUT implementation combines architectural styles, object-oriented design patterns, and framework-supported mechanisms.

The following table summarizes their application and motivation. Detailed class structures and runtime interactions for the State and Strategy patterns are already presented in the Class View and Runtime View.

Style or pattern	Application in NightOUT	Motivation
Client-server	A React and TypeScript client communicates with a Java Spring Boot backend through REST APIs and JSON DTOs.	Separates presentation and navigation from application logic and persistent data management.

Style or pattern	Application in NightOUT	Motivation
Layered architecture	The normal backend dependency path is Controller → Service → Focused Data Mediator → Repository → H2 Database.	Supports separation of concerns, clearer responsibilities, and independent testing of application services.
Repository	Spring Data JPA repositories provide persistent access to users, venues, events, tickets, pregames, friendships, promotions, and notifications.	Abstracts database operations and avoids embedding persistence queries in application services.
Data Transfer Object	Request and response DTOs are exchanged at the REST boundary and constructed through mapping components.	Prevents direct exposure of JPA entities, internal fields, and bidirectional persistence relationships.
State	The ticket lifecycle is represented by concrete implementations of `TicketState` for pending, confirmed, waiting-list, cancelled, and expired tickets.	Encapsulates status-dependent operations and makes invalid ticket transitions explicit.
Simple Factory	`TicketStateFactory` selects the runtime State corresponding to the persistent `TicketStatus`.	Centralizes State creation and prevents `TicketService` from directly constructing concrete State objects.
Observer-style events	Ticket and friendship services publish Spring application events consumed by notification listeners. Pregame notifications currently call `NotificationService` directly.	Reduces coupling between domain operations and notification creation while using Spring-managed listener registration.
Strategy	`RecommendationEngine` evaluates multiple implementations of `RecommendationStrategy`, including music preference, friends attending, distance, booking history, saved events, and popularity.	Allows recommendation criteria to be extended independently and combines multiple score contributions.
Focused data mediation	Domain-specific mediators coordinate repository access for users, events, tickets, pregames, friendships, recommendations, notifications, and dashboards.	Limits service dependencies and avoids creating one global data-access object.

The architecture is predominantly layered. `AuthenticationService` directly accesses `AppUserRepository`, while `NightOutMapper` performs additional repository reads when constructing aggregate event DTOs.

Some elements proposed in the initial design were implemented differently. Event filtering uses conditional operations rather than Strategy objects, event popularity is stored rather than dynamically calculated, and persistence coordination is distributed among focused mediators rather than centralized in one global mediator.

3 User Interface Design

This section provides an overview of the NightOUT user interface and its organization across the three supported application roles. The interface is implemented as a mobile-oriented React and TypeScript web application backed by the Spring Boot REST API.

3.1 UI Overview

NightOUT adopts a mobile-oriented interface designed around quick access to nightlife events and role-specific operations.

The normal-user interface uses card-based layouts for event discovery, recommendations, tickets, and pregame rooms. Event cards highlight the most relevant information, including venue, date, genre, price, availability, and social indicators.

The application provides separate interfaces for normal users, venue managers, and PR managers. After login, users are redirected to the appropriate route group according to their role.

Operational information is retrieved from the backend through the shared REST client. The frontend does not maintain an independent domain dataset. Pages provide explicit loading, error, empty-result, and operation-status feedback when interacting with backend services.

A lightweight browser session marker preserves role-based navigation after page refresh. It supports frontend routing but does not represent a server-side authenticated session.

3.2 Navigation and Role-Based Routing

Frontend navigation is implemented through React Router. The public login page redirects the user to a role-specific interface according to the role returned by the backend:

- `NORMAL_USER` → `/feed`
- `VENUE_MANAGER` → `/manager`
- `PR_MANAGER` → `/pr`

Frontend route guards restrict access to pages that are not associated with the active role. The main interface groups are summarized below.

Role	Main interfaces
Normal user	Event feed and recommendations, event details, ticket checkout and ticket list, pregame rooms, profile, friendships, and notifications
Venue manager	Dashboard, event list and creation, promotion management, venue profile, and ticket inspection
PR manager	Dashboard, attributed tickets, and PR account

Secondary normal-user interfaces include privacy settings, demo payment methods, help and support, and avatar management. These pages reuse the same role navigation and REST-backed interaction model and are not represented separately in this overview.

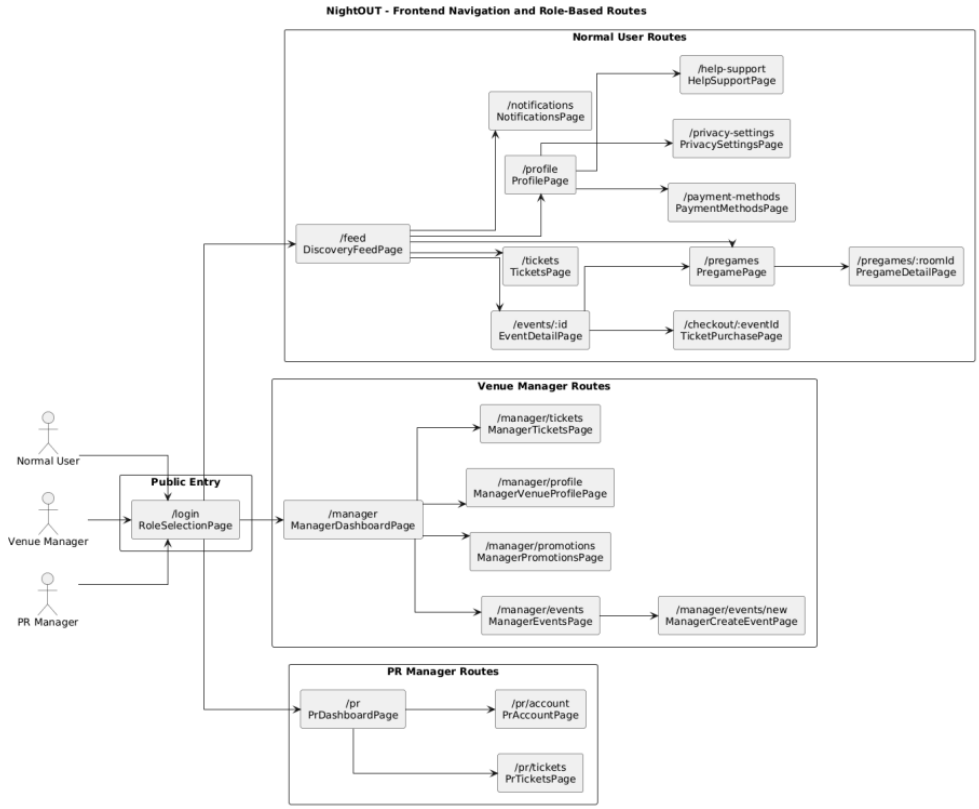
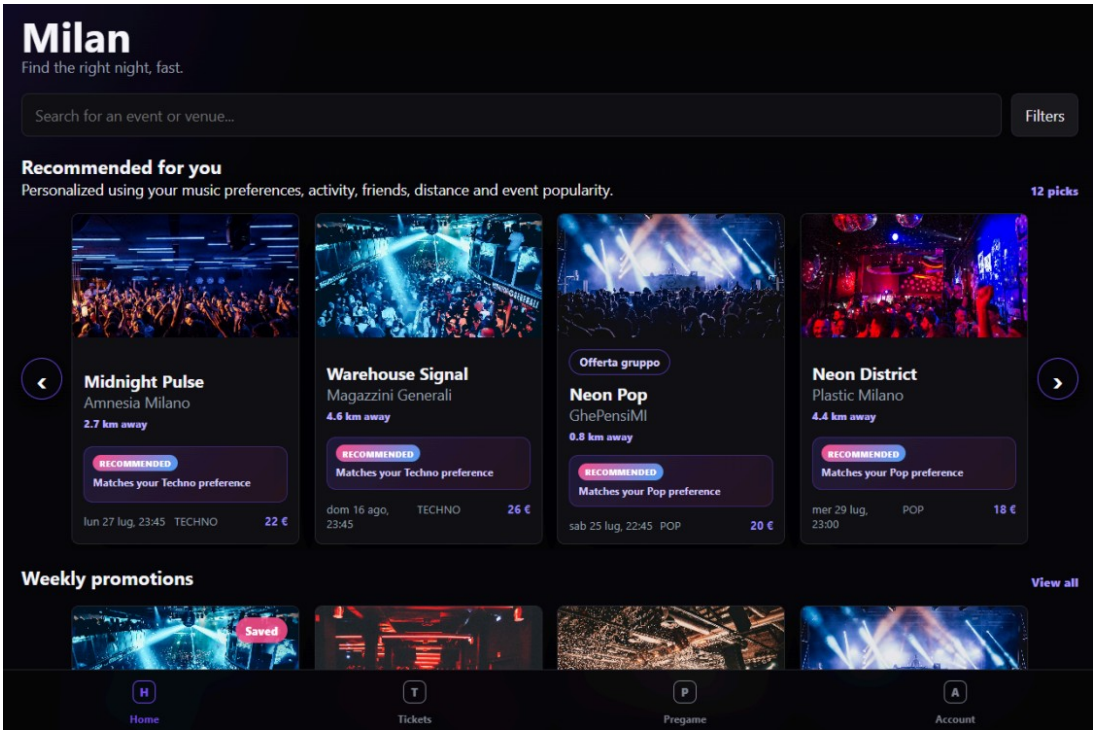


Figure 10: NightOUT - Frontend Navigation and Role-Based Routes

3.3 Representative Interfaces

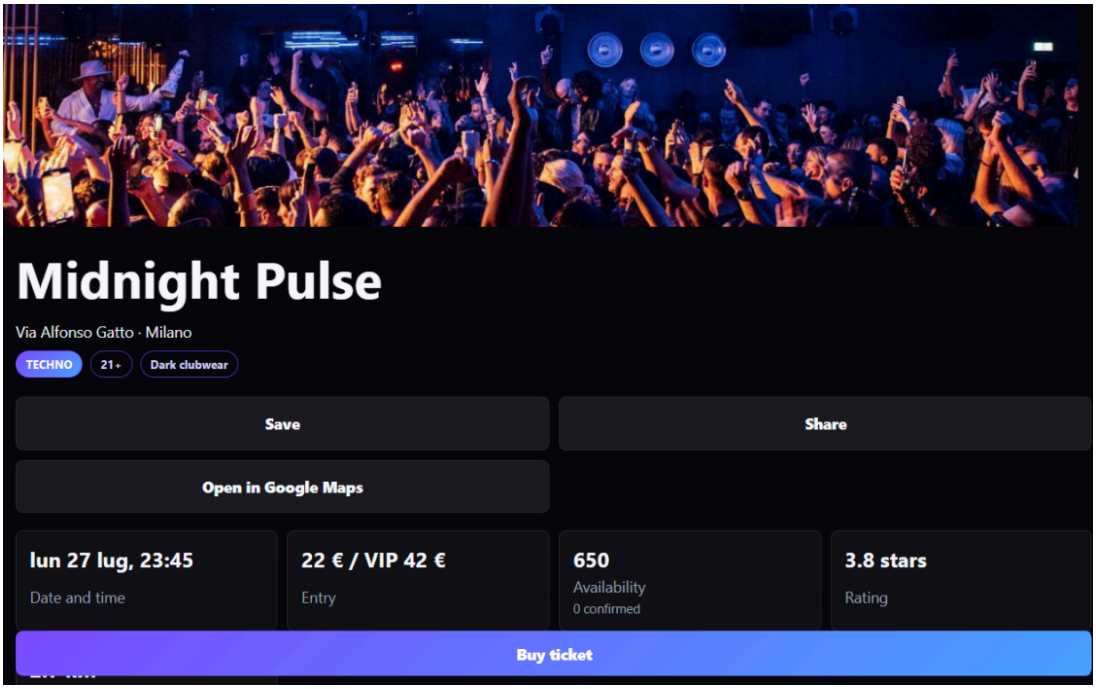
The following screenshots show the most representative NightOUT interfaces. They illustrate the event-centred mobile design, the main normal-user workflows, and the dedicated management views available to venue and PR managers.

3.3.1 Discovery Feed



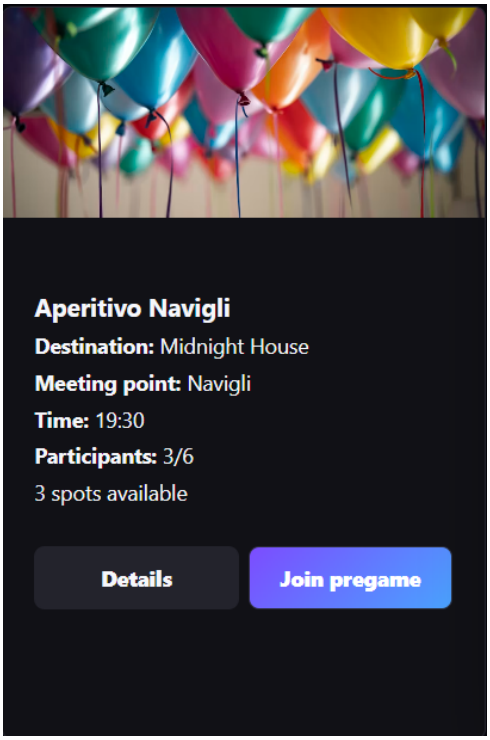
Normal users browse events, apply search and filtering criteria, and access personalized recommendations.

3.3.2 Event Detail and Ticket Action



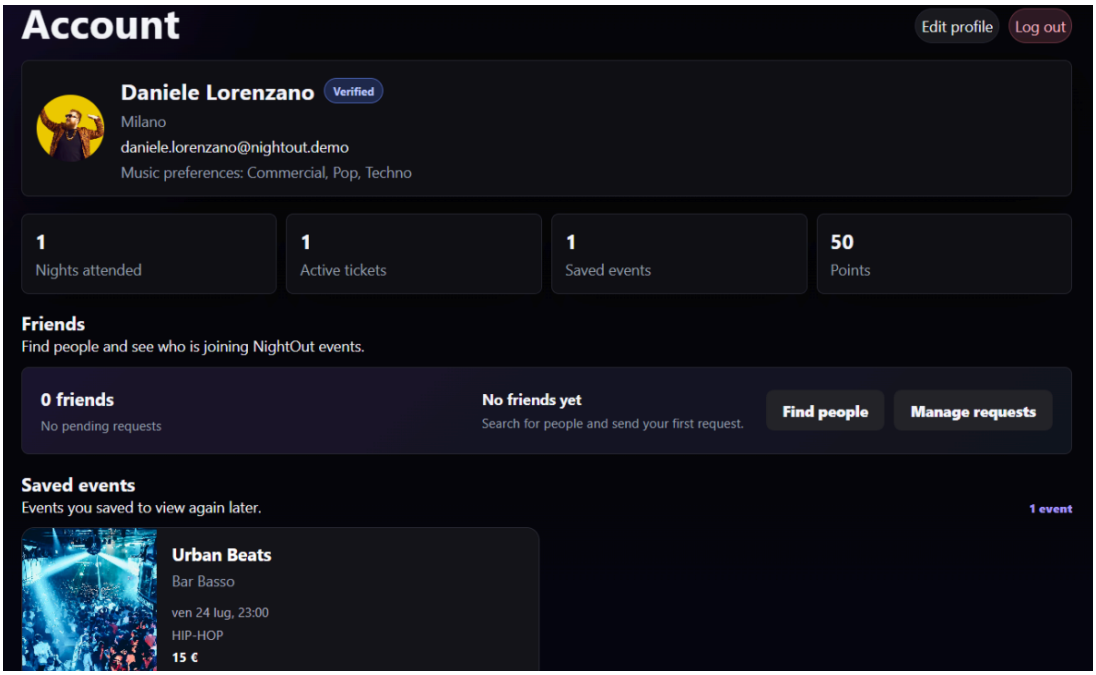
The interface combines venue and event information with availability, promotions, social indicators, and access to the ticket workflow.

3.3.3 Pregame Room



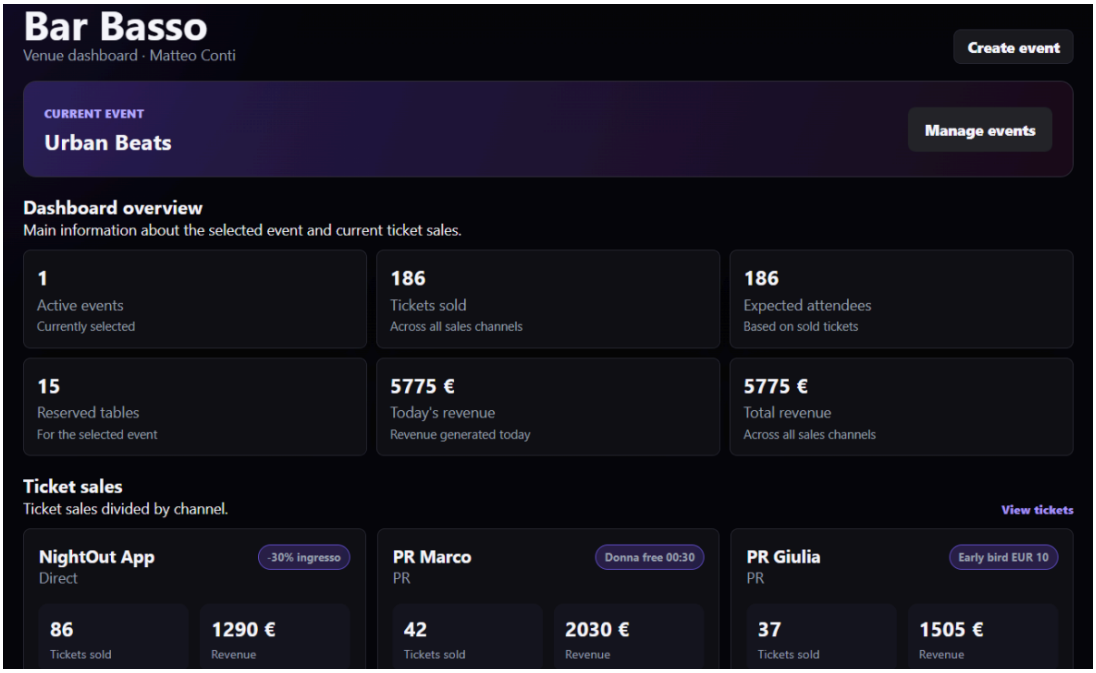
Users can inspect the linked event, meeting information, available capacity, and current participants before joining or leaving the room.

3.3.4 User Profile



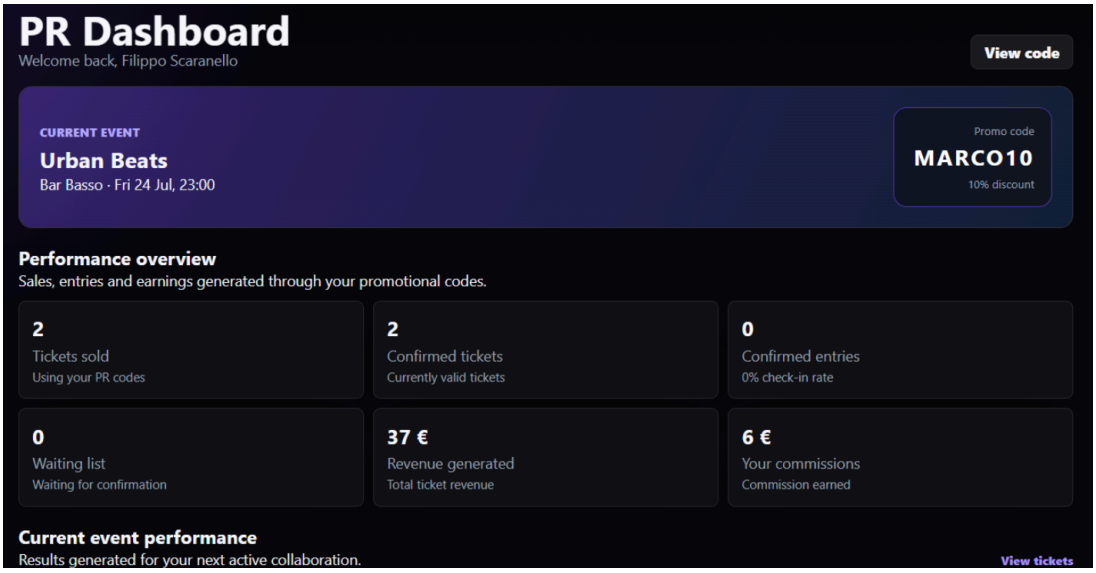
The profile groups personal information, music preferences, saved events, friendships, and access to account-related settings.

3.3.5 Venue Manager Dashboard



Venue managers access event, ticket, pregame, and sales-related information through a dedicated management interface. The manager interface provides also a structured form for creating events associated with managed venues.

3.3.6 PR Dashboard



PR managers inspect assigned events, attributed ticket activity, and commission-related indicators through a dedicated role-specific view.

4 Requirements Traceability

4.1 Traceability Approach and Status Definitions

This section maps the requirements defined in the Requirements Analysis and Specification Document to the corresponding design and implementation elements of NightOUT.

The prefixes used in the traceability matrices are:

- FR-US: User and Profile Requirements;
- FR-EV: Event Discovery Requirements;
- FR-REC: Recommendation Requirements;
- FR-RS: Reservation and Ticketing Requirements;
- FR-PR: Pregame Requirements;
- FR-SO: Social Layer Requirements;
- FR-NO: Notification Requirements;
- FR-VM: Venue Manager Requirements;
- NFR-USAB: Usability Requirements;
- NFR-PERF: Performance Requirements;
- NFR-REL: Reliability Requirements;
- NFR-ROB: Robustness Requirements;
- NFR-MAIN: Maintainability Requirements;
- NFR-SEC: Security and Privacy Requirements.

Traceability is evaluated against the current implementation baseline. The matrix distinguishes between functionality that is completely available, functionality implemented through a design different from the initial proposal, partially implemented functionality, mock or seed-only behavior, functionality not implemented, and requirements that have not yet been verified through appropriate tests.

4.2 Functional Requirements Traceability

4.2.1 User and Profile Requirements

Requirement	Main design elements	Status
FR-US-01 — User login	Login UI, `AuthController`, `AuthenticationService`	Different
FR-US-02 — Profile visualization	`ProfilePage`, `UserController`, `UserService`	Implemented
FR-US-03 — User preferences	`ProfilePage`, `UserService`, `AppUser`	Implemented
FR-US-04 — Save events	`EventDetailPage`, `UserService`, `EventParticipation`	Implemented
FR-US-05 — View saved events	`ProfilePage`, saved-events API, `UserService`	Implemented

4.2.2 Event Discovery Requirements

Requirement	Main design elements	Status
FR-EV-01 — Browse events	`DiscoveryFeedPage`, `EventController`, `EventService`	Implemented
FR-EV-02 — View event details	`EventDetailPage`, `EventService`, `NightOutMapper`	Implemented
FR-EV-03 — Complete event information	`EventDetailPage`, `Event`, `Venue`	Implemented
FR-EV-04 — Filter events	`DiscoveryFeedPage`, `EventService`	Different
FR-EV-05 — Empty filter result	`DiscoveryFeedPage`	Implemented
FR-EV-06 — Event popularity	`Event.popularityScore`, `EventService`	Partial

4.2.3 Recommendation Requirements

Requirement	Main design elements	Status
FR-REC-01 — Recommended event feed	`DiscoveryFeedPage`, `RecommendationService`, `RecommendationEngine`	Implemented
FR-REC-02 — Music-based recommendation	`MusicPreferenceStrategy`, user preferences	Implemented
FR-REC-03 — Social recommendation	`FriendsAttendingStrategy`, friendships, confirmed tickets	Implemented
FR-REC-04 — Distance recommendation	`DistanceStrategy`, `GeographicDistanceService`	Implemented

4.2.4 Reservation and Ticketing Requirements

Requirement	Main design elements	Status
FR-RS-01 — Request reservation	`TicketPurchasePage`, `TicketController`, `TicketService`	Implemented
FR-RS-02 — Confirm with available capacity	`TicketService`, `TicketStateFactory`, `PendingTicketState`	Implemented
FR-RS-03 — Waiting list for full events	`TicketService`, `PendingTicketState`	Implemented
FR-RS-04 — View reservation status	`TicketsPage`, `TicketDto`, `TicketService`	Implemented
FR-RS-05 — Cancel reservation	`TicketService`, `TicketState`	Implemented
FR-RS-06 — Promote waiting-list ticket	`TicketService`, `WaitingListTicketState`	Implemented
FR-RS-07 — Prevent duplicate active reservations	`TicketService`, `TicketRepository`	Implemented
FR-RS-08 — Reservation status values	`TicketStatus`, `TicketState`	Implemented
FR-RS-09 — Digital confirmation	`Ticket.qrPayload`, `TicketDto`, `TicketsPage`	Implemented

4.2.5 Pregame Requirements

Requirement	Main design elements	Status
FR-PR-01 — Create pregame room	`PregamePage`, `PregameController`, `PregameService`	Implemented
FR-PR-02 — Pregame room information	`PregameRoom`, `PregameRoomDto`, `PregameDetailPage`	Implemented
FR-PR-03 — Browse pregame rooms	`PregamePage`, `PregameController`, `PregameService`	Implemented
FR-PR-04 — Join pregame room	`PregameService`, `PregameDataMediator`	Implemented
FR-PR-05 — Prevent joining full rooms	`PregameService`	Implemented
FR-PR-06 — Leave pregame room	`PregameService`	Implemented
FR-PR-07 — Prevent duplicate participation	`PregameService`	Implemented

4.2.6 Social Layer Requirements

Requirement	Main design elements	Status
FR-SO-01 — Friendship creation	Friends Section, `FriendshipController`, `FriendshipService`	Different
FR-SO-02 — View friends	FriendsSection, `FriendshipService`	Implemented
FR-SO-03 — Event participation status	`EventParticipation`, `UserService` , tickets	Partial
FR-SO-04 — Friends attending events	`EventDetailPage`, event service, confirmed tickets	Partial
FR-SO-05 — Activity feed	-	Not implemented

4.2.7 Notification Requirements

Requirement	Main design elements	Status
FR-NO-01 — Reservation notifications	Ticket events, ticket observer, `NotificationService`	Implemented
FR-NO-02 — Waiting-list notifications	Promotion event, ticket observer, `UserNotification`	Implemented
FR-NO-03 — Friend-event notification	Notification type, seeded notification	Partial
FR-NO-04 — Event-update notification	`NotificationType.EVENT_UPDATE`	Not implemented
FR-NO-05 — View notifications	`NotificationsPage`, `NotificationController`, `NotificationService`	Implemented
FR-NO-06 — Mark notification as read	`NotificationsPage`, `NotificationService`, `UserNotification`	Implemented

4.2.8 Venue Manager Requirements

Requirement	Main design elements	Status
FR-VM-01 — Venue-manager login	Login UI, `AuthController`, `AuthenticationService`	Different
FR-VM-02 — Create event	`ManagerCreateEventPage`, `VenueManagementService`, `EventRepository`	Partial
FR-VM-03 — Update event	`ManagerEventsPage`, `VenueManagementService`	Partial
FR-VM-04 — Manage event capacity	Manager event form, `VenueManagementService`	Implemented
FR-VM-05 — Manage promotions	`ManagerPromotionsPage`, `PromotionController`, `PromotionService`	Implemented
FR-VM-06 — View event statistics	`ManagerDashboardPage`, `DashboardService`, `DashboardDataMediator`	Partial

4.3 Traceability Summary

Status	Main areas
Implemented	Profiles, saved events, event discovery, recommendations, ticket lifecycle, pregames, core notifications, promotions
Different	Authentication and session management, event filtering, Ticket instead of Reservation, friendship workflow
Partial	Event popularity, social participation, friend-event notifications, venue ownership, manager statistics, security and reliability
Not implemented	Social activity feed, event-update notifications
Not verified	Performance targets and concurrent ticket operations

5 Implementation, Integration and Test Plan

This section summarizes the dependency-based implementation order, the integration of the main NightOUT components, the current verification status, and the test data used by the prototype.

The application is already implemented and runnable. Therefore, the implementation order represents the logical dependencies among components rather than the exact chronological order of repository commits. The testing plan distinguishes between verification already performed and the principal scenarios still requiring automated coverage.

5.1 Implementation Order

NightOUT follows an incremental implementation approach. Foundational infrastructure and persistent domain entities are required before application services and frontend use cases can be completed.

The current implementation can be organized into the following dependency-based phases.

Phase	Main components	Dependencies
1 — Infrastructure and persistence	Spring Boot and React projects, JPA, H2, repositories, DTOs, exception handling, startup data	-
2 — Users and accounts	`AppUser`, roles, authentication, profiles, preferences, frontend session and role routing	Phase 1
3 — Venues and events	Venues, events, event discovery, filtering, event details and manager event services	Phase 2
4 — Core operational workflows	Ticket lifecycle, capacity management, waiting list, notifications and pregame rooms	Phase 3
5 — Social and recommendation features	Saved events, friendships, friends attending and recommendation strategies	Phases 2–4
6 — Role extensions and stabilization	Venue-manager and PR interfaces, promotions, dashboards, build verification and documentation alignment	Previous phases

The implementation uses Ticket instead of the initially proposed Reservation entity. Event filtering is implemented through conditional service operations, while personalized recommendation scoring uses the Strategy pattern. Remaining stabilization priorities concern backend authorization, manager venue-ownership checks, dashboard consistency, frontend lint errors, and broader automated test coverage.

5.2 Integration Plan

Integration followed the architectural dependency direction from persistence to application services and frontend workflows. The current status of the main integration areas is summarized below.

Integration area	Integrated elements	Current status
Persistence	JPA entities, Spring Data repositories and H2 database	Operational; data is recreated at backend startup
Services and data access	Application services, focused data mediators and repositories	Operational; `AuthenticationService` and `NightOutMapper` are limited direct-repository exceptions
REST boundary	Controllers, request and response DTOs, validation and exception handling	Operational for the principal user, event, ticket, pregame, social, manager and PR operations
Frontend and backend	React pages, shared Axios client and Spring Boot REST APIs	Operational; the frontend production build succeeds
Ticket workflow	Ticket State objects, capacity checks, waiting list, Spring events and notifications	Operational for sequential prototype use
Pregame and social workflow	Pregame membership, friendships, saved events and friends attending	Operational; some social behavior remains partial
Recommendations	User, event, ticket, saved-event and friendship data with recommendation strategies	Operational; social contributions depend on live friendship records
Manager and user views	Manager-created events and normal-user discovery use shared persistence	Operational with known ownership, dashboard-identity and navigation defects

Cross-component data remains visible to all interfaces while the backend is running. For example, manager-created events become available in the normal-user feed, confirmed tickets affect event availability, and accepted friendships contribute to social event information and recommendations.

5.3 Testing Strategy

The verification strategy combines automated backend testing, build and static checks, and manual runtime verification. The automated test suite currently focuses on the most critical backend components: application startup, authentication, ticket business rules, scheduled ticket transitions, ticket lifecycle constraints, and the State and Factory patterns used to manage ticket statuses.

The implemented tests include both integration and unit testing. Integration tests verify the collaboration between the Spring application context, REST controllers, services, repositories, and the H2 database. Unit tests isolate the component under test by replacing external dependencies with Mockito mocks.

The test cases were designed according to both black-box and white-box principles. Black-box tests derive expected results from functional requirements, such as valid authentication, invalid credentials, duplicate-ticket prevention, and restrictions after an event has started. White-box tests exercise selected internal branches, State transitions, Factory outputs, and component interactions.

Boundary Value Analysis is applied to the ticket confirmation deadline by testing the system immediately before the deadline, exactly at the deadline, and immediately after it. No coverage percentage is reported because no automated coverage tool is currently configured.

5.3.1 Tests Currently Implemented

The backend currently includes 28 automated tests, grouped into the following test classes.

Test class	Type	Verified behavior	Tests	Result
`BackendApplicationTests`	Spring smoke/integration	Application context startup, dependency configuration, and startup data loading	1	Passed
`AuthControllerTests`	MockMvc integration and black-box	Valid authentication for all roles, invalid credentials, unknown users, HTTP responses, and exclusion of passwords from API responses	6	Passed
`TicketExpirationSchedulerTests`	Unit and interaction	The scheduled component delegates automatic ticket transitions to the ticket service	1	Passed
`TicketServiceTests`	Unit and white-box	Rejection of requests after event start, duplicate active-ticket prevention, and execution of automatic transition checks for all ticket categories	3	Passed
`TicketEntityTests`	Unit, branch, and boundary testing	Initial ticket state, confirmation deadline, valid and invalid transitions, final states, active-state evaluation, and exact deadline behavior	6	Passed
`TicketStateFactoryTests`	Unit and white-box	Correct State implementation returned for every ticket status and rejection of null input	6	Passed
`TicketStateBehaviorTests`	Unit and State-pattern testing	Allowed and forbidden operations for PENDING,	5	Passed

5.3.2 Main Tested Behaviors

The automated test suite verifies the following groups of behavior:

- correct authentication and role recognition for normal users, venue managers, and PR managers;
- rejection of invalid passwords and unknown email addresses;
- exclusion of sensitive password data from authentication responses;
- successful startup of the complete Spring application context;
- correct delegation from the automatic scheduler to the ticket service;
- rejection of ticket requests after an event has started;
- prevention of duplicate active tickets for the same user and event;
- execution of automatic checks for PENDING, WAITING_LIST, and CONFIRMED tickets;
- mandatory PENDING initial state for newly created tickets;
- automatic creation of the 15-minute confirmation deadline;
- valid and invalid ticket-state transitions;
- immutability of terminal CANCELLED and EXPIRED states;
- correct creation of State objects through the Factory Pattern;
- ticket expiration immediately before, exactly at, and immediately after the confirmation deadline.

Assertions such as `assertEquals`, `assertThrows`, `assertTrue`, `verify`, HTTP-status checks, and JSON-path checks are used as automated test oracles.

5.3.3 Build and Test Status

Verification	Result	Notes
Backend compilation and automated tests	Passed	The backend compiles and all 28 automated tests pass with no failures, errors, or skipped tests
Windows Maven Wrapper	Passed	The complete test suite can be executed through `.\mvnw test`
Spring application startup	Passed	The application context, dependencies, repositories, and startup data loader are initialized correctly
Frontend production build	Passed	TypeScript compilation and Vite production bundling succeed
Frontend lint	Failed	The current frontend lint execution reports 27 errors and 1 warning
Frontend automated tests	Not available	No dedicated frontend test command or automated component-test suite is currently configured
Coverage measurement	Not available	No coverage tool, such as JaCoCo, is currently configured; therefore, no line or branch coverage percentage is claimed

The successful Maven execution produces the following summary:

```
Tests run: 28, Failures: 0, Errors: 0, Skipped: 0
BUILD SUCCESS
```

5.3.4 Relation to Verification and Validation Theory

The implemented suite mainly supports software verification, since it checks whether the implementation is consistent with the specified business rules and architectural design.

The authentication tests are primarily black-box tests because they send HTTP requests and verify externally observable responses without depending on the controller implementation. The ticket entity, service, scheduler, State, and Factory tests also include white-box aspects because they are derived from the internal structure of the code, including branches, switch cases, component interactions, and allowed State transitions.

The tests operate at different levels:

- **Unit level**: Ticket entity, Ticket service, Scheduler, State implementations, and State Factory;
- **Integration level**: authentication REST API and complete Spring application context;

- **Boundary testing**: ticket confirmation deadline;
- **Regression testing**: previously verified behavior can be automatically checked after future modifications.

The test suite does not prove the absence of every possible defect. Instead, it verifies representative and critical cases with a high likelihood of exposing errors in the main business rules.

5.3.5 Remaining Automated Testing Plan

Although the ticket lifecycle and authentication subsystems now have automated coverage, other application areas are still verified mainly through manual execution.

Test level	Remaining target	Representative examples
Unit	Pregame, recommendation, promotion, commission, friendship, and manager services	Pregame capacity, recommendation-score contributions, promotion validity, commission calculation, friendship transitions, and venue ownership
Integration	Controller-service-mediator-repository collaborations beyond authentication	Ticket creation through REST, ticket confirmation, pregame join, friendship request, event creation, and notification persistence
Frontend	Components, routing, forms, and API-driven states	Login feedback, loading states, empty states, error handling, ticket rendering, and manager-form validation
End-to-end	Complete user journeys across frontend and backend	Ticket purchase, waiting-list promotion, pregame membership, manager event creation, and recommendation retrieval
Concurrency and robustness	Simultaneous operations and malformed requests	Capacity protection, duplicate active tickets, single waiting-list promotion, and concurrent confirmation requests

Concurrent ticket operations require additional verification. The current tests verify the defined rules during sequential execution, but no automated test currently proves that simultaneous requests cannot exceed event capacity, create duplicate active tickets, or promote multiple waiting-list tickets for a single available place.

5.4 Test Data

NightOUT uses a realistic synthetic startup dataset loaded from `nightout-workbook-seed.json` and additional records created programmatically by `DemoDataLoader`.

The dataset includes normal users, venue managers, PR managers, venues, events, tickets, pregames, promotions, notifications, sales information, and supporting entities. It provides

scenarios with different event capacities, ticket statuses, user roles, prices, music genres, locations, and recommendation inputs.

Operational data is stored in an in-memory H2 database configured with Hibernate `create-drop`. The data is recreated whenever the backend starts, remains shared across all frontend interfaces during execution, and is removed when the backend stops or restarts. Avatar files are stored separately in the local file system.

Test fixtures include:

- users with different application roles;
- valid and invalid authentication credentials;
- future and already-started events;
- duplicate active-ticket scenarios;
- PENDING, CONFIRMED, WAITING_LIST, CANCELLED, and EXPIRED tickets;
- ticket confirmation deadlines before, at, and after the boundary value;
- mocked mediator, mapper, event-publisher, and service dependencies.